

Avoiding Blocking by Scheduling Transactions using Quantum Annealing

Tim Bittner

Sven Groppe

tim.bittner96@gmx.de

groppe@ifis.uni-luebeck.de

Institute of Information Systems (IFIS), University of Lübeck
Lübeck

ABSTRACT

Quantum annealers are a special kind of quantum computers for solving optimization problems. In this paper, we investigate the benefits of quantum annealers in the field of transaction synchronization. In particular, we show how transactions using the 2-phase-locking protocol can be optimally distributed to any number of available machines to reduce transaction waiting times. Therefore an instance of the problem will be transformed into a formula that is accepted by quantum annealers. In an experimental evaluation, the runtime on a quantum annealer outperforms the runtime of traditional algorithms to solve combinatorial problems like simulated annealing already for small problem sizes.

CCS CONCEPTS

• Information systems → Data management systems; Database transaction processing.

KEYWORDS

Quantum computing, quantum annealing, transaction processing, synchronization, 2-phase-locking, database, schedule, D-Wave

ACM Reference Format:

Tim Bittner and Sven Groppe. 2020. Avoiding Blocking by Scheduling Transactions using Quantum Annealing. In *24th International Database Engineering & Applications Symposium (IDEAS 2020)*, August 12–14, 2020, Seoul, Republic of Korea. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3410566.3410593>

1 MOTIVATION

In the last decade quantum computing including special forms like quantum annealing solving quadratic unconstrained binary optimization (QUBO) problems [2, 12] has become more and more popular and the range of applications wider and wider [15, 17, 20, 21], leading to a lot of time and money being spent on research in this area. Many scientific papers deal with the comparison of standard algorithms and quantum algorithms [7] and quite a few

of them belong to the field of optimization problems [22, 24, 25]. There are only few contributions dealing with optimizing database problems with quantum computers like query optimization [24].

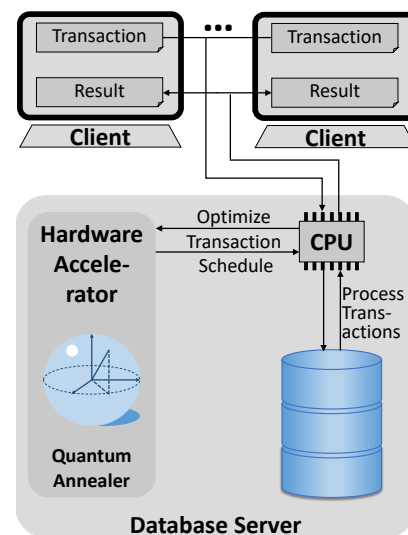


Figure 1: Hardware accelerating transaction schedules via quantum annealing

The field of transaction synchronization includes many combinatorial problems and hence offers various problems and certainly plenty of potential for optimizations by quantum computers. This work focuses in particular on the optimization of the 2-phase-locking protocol for transactions, which prevents conflicts between transactions by *preclaiming* all necessary locks at begin of transaction and avoids cascading aborts by holding all locks until the end of transaction, but forces waiting times due to the locks. Our contribution deals with avoiding waiting times and finding an optimal execution sequence. Thereby a solution approach for quantum annealers used as hardware accelerator for database tasks [14] is derived (see Figure 1), which are designed to solve hard optimization problems [1]. Since inputs of quantum annealers cannot be arbitrary, some preprocessing is required to transform given problems in such a way that processing by a quantum annealer is possible. After preprocessing, quantum annealers solve the problem in constant time in contrast to other hardware accelerators [13]. Our work is inspired by work on job shop scheduling problems (being among the hardest combinatorial optimization problems [11]) for quantum

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

IDEAS 2020, August 12–14, 2020, Seoul, Republic of Korea

© 2020 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-7503-0/20/06...\$15.00

<https://doi.org/10.1145/3410566.3410593>

computers [25], but transaction scheduling additionally needs to consider conflicts between transactions.

Our main contributions are

- proposing a formula runnable on quantum annealers for an optimal scheduling of transactions synchronized by 2-phase-locking protocol with preclaiming of all locks at begin of transaction and holding all locks until end of transaction,
- a complexity analysis of the preprocessing time and the number of required qubits, and
- an extensive experimental evaluation comparing the runtimes of quantum annealers with the simulated annealing algorithm.

We introduce the basics about transactions, transaction management and quantum computers in Section 2. Section 3 covers the main part of the work, the formal model for the problem and the transformation from this model into a formula suitable for a quantum annealer. We provide a short insight into the implementation and the used software in Section 4. We estimate the number of required qubits, and analyze the complexity of preprocessing and experiments on a quantum annealer in Section 5. We finally summarize and provide a small outlook for future work in Section 6.

2 BASICS

This section is dedicated to the basics of transaction management (see Section 2.1) and quantum computers (see Section 2.2) with special focus on quantum annealing.

2.1 Transaction Management

A **transaction** $t = \langle s_1, \dots, s_n \rangle$ of length n is a series of operations s_i , carried out by a single user or application program, which reads or updates the contents of the database [8]. Each operation $s_i = a_i(e_i)$ consists of the type of access $a_i \in \{r, w\}$, where r represents a read access and w a write access, and the object e_i to be accessed.

For example, the transaction $t = \langle r(A), w(A), r(B), w(B) \rangle$ is of length $|t| = 4$.

2.1.1 Conflicts between Transactions: Since transactions often need to access a database at the same time and thereby often reference the same objects, transactions in a database system are required to fulfill the so-called ACID properties [8]. The focus here is on the fulfillment of the (I)solation property, which guarantees to avoid problems of unsynchronized parallel execution of several transactions and requires concurrently executed transactions to not influence each other.

2.1.2 Conflict Management: To ensure the isolation property, conflicts between transactions must be dealt with. The simplest strategy to deal with conflicts would be to execute the transactions serially. Since this is too slow for operational systems and also many transactions do not conflict with each other at all, in practice transactions are executed in parallel. There are various approaches to dealing with the conflicts that arise, one of them is the use of locks. For this purpose, each transaction acquires a lock for an object before access and releases it after access, thus preventing concurrent access to an object.

The **two-phase-locking protocol** is a locking protocol that requires each transaction consisting of two subsequent phases, the locking phase and the release phase. During the locking phase, the

transaction may acquire locks but not release them, whereas the release phase requires the release of previously required locks. If a transaction has released a lock, it may not acquire any new ones. There is a distinction between *read locks* (also called *shared locks*) and *write locks* (also called *exclusive locks*). Variants of the protocol include the conservative two-phase-locking protocol and the strict two-phase-locking protocol. The conservative variant (*preclaiming*) requires that all locks that are required during a transaction are acquired before the transaction is started.¹ The strict variant holds all locks until the end of a transaction, which avoids *cascading aborts* occurring in case of so called *dirty reads* of objects written by transactions, which are later aborted resulting in an abort of the current transaction as well. In this contribution, we consider the combination of the conservative and the strict two-phase-locking protocol in our transaction model, which results in a serial execution of all transactions that access the same objects.

2.2 Quantum computer

Quantum computers are computers that are not based on classical mechanics, instead they exploit the effects of quantum mechanics.

2.2.1 Basic idea: Quantum mechanics describes the states and behavior of particles that are smaller than the size of an atom and do not follow the laws of classical physics. At this scale, there occur effects that the quantum computer makes use of, especially the principle of superposition and that of quantum entanglement. The quantum computer uses qubits, which can take on 2 states simultaneously due to the principle of superposition. While a bit can assume the state 0 or 1, a qubit assumes the states 0 and 1 simultaneously. If a measurement of the state is made, the qubit changes to one of the two states. Both states have relative probabilities with which they are assumed in the measurement. The principle of quantum entanglement enables the mutual influence of qubits, since entangled qubits mutually influence their probabilities. Imagining the principle of superposition as a special form of parallel computing opens up a new world of computation beyond polynomial time and allows in theory an exponential speedup compared to classical computers.

2.2.2 Quantum computing: finds desired solutions of a problem by clever manipulation of single qubits as well as entangled qubits. For the purpose of manipulating qubits, gates provide elementary operations on one or two qubits. For example, the Hadamard-Gate puts one qubit into superposition and a Controlled-NOT(CNOT)-Gate inverts a second qubit depending on the first [6]. A quantum computer is thus able to execute quantum algorithms like Shor's algorithm for factorizing large numbers [21] or Grover's algorithm for searching in huge unsorted databases [15, 16].

2.2.3 Quantum Annealers: are a special form of quantum computers [10, 18], which are designed to solve hard optimization problems, but cannot execute quantum algorithms like Shor's algorithm. Quantum annealers find the global minimum of a given objective function by transforming a simplified objective function

¹Please note that for those transactions, for which the required locks are not known before processing, the required locks can be determined by an additional phase before transaction processing. The contribution in [23] describes such an approach which can be also applied in our scenario.

with known global minimum into the objective function of interest, so that the quantum annealer always remains in the state of the global minimum, which represents the desired result. The most widely-known quantum annealers are manufactured by the canadian company D-Wave [1], which produced the first commercial quantum computers.

Input format: Quantum annealers have very limited input formats, such that all problems have to be transformed into this input format before being solved and transformed back after annealing. Quantum annealers solve quadratic unconstrained binary optimization (QUBO) problems [2, 12], which belong to the class of NP-hard problems. A QUBO-problem is defined by N weighted binary variables $X_1, \dots, X_N \in \{0, 1\}$, either as linear or quadratic term:

$$\sum_{0 < i \leq N} w_i X_i + \sum_{i < j \leq N} w_{ij} X_i X_j, \text{ where } w_i, w_{ij} \in \mathbb{R}$$

The quantum annealer transforms these weightings into energy levels of single qubits or between two qubits, and by minimizing the energy level the quantum annealer finds a minimization of the objective function, in other words a variable assignment for the QUBO-problem. The idea of quantum annealing is very close to the simulated annealing approach [19] on traditional computers (see Figure 2).

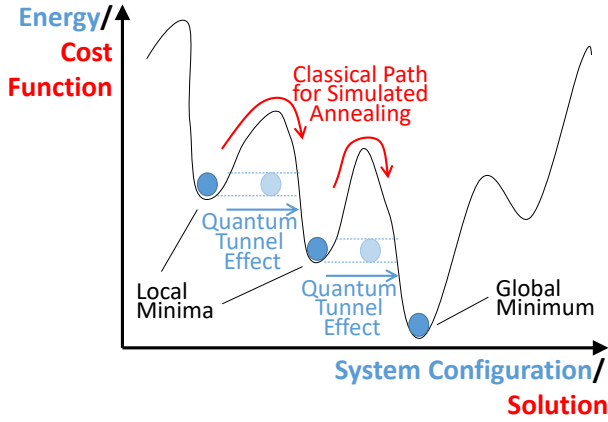


Figure 2: Simulated annealing (sa) [19] versus quantum annealing (qa) to reach a global minimum (simplified discussion): Quantum tunneling driven by an external magnetic field “passes” high energy/cost function “peaks” instead of climbing them as in sa. Larger tunnels can be passed (qa) and bigger peaks climbed (sa) with a high magnetic field (qa) and a high temperature (sa). Then lower magnetic fields (qa) and lower temperatures (sa) hinders to get out of a valley of the energy (qa) and cost function (sa) respectively with a global minimum. As long as the function fits into the system configuration size, qa determines the minimum in constant time in contrast to sa, which takes longer times for more complex functions.

3 MODELLING

In this section, we describe the formal model of instances of the considered scheduling problem (see Section 3.1) and afterwards formulate the scheduling problem as QUBO-problems (see Section 3.2) for valid solutions (see Section 3.3) and optimal solutions (see Section 3.3.1), and optimize the total solution further (see Section 3.3.2).

3.1 Formal model

The goal of this optimization is to reduce, or in the best case completely eliminate, the waiting times of transactions that use the two-phase-locking protocol. A transaction must wait whenever it requests a lock, but this lock is held by another transaction. In this case, one transaction blocks the other and only one of the two can be executed at a time. For simplicity of presentation, we consider the conservative and strict two-phase-locking protocol². We assume that k machines are available for processing n transactions. Thus k transactions can run parallel to each other, if they do not block each other. We hence consider the scheduling problem, where n transactions have to be distributed to k machines, taking into account that two transactions blocking each other are never executed simultaneously. Thereby the maximum execution time R (see Section 3.3.3) should be minimized over all machines. An instance of this scheduling problem consists of a set T of transactions with $|T| = n$, a set M of machines with $|M| = k$ (see Figure 6) and a set $O \subseteq T \times T$ of blocking transactions (see Figure 5). Each transaction $t_i \in T$ has a certain length l_i (see Figure 3) and thus an upper bound $r_i = R - l_i$ for its start time (see Figure 4).

3.1.1 Running Examples. This paper contains two running examples:

Example E_1 : The first running example E_1 deals with a rather complex scenario and contains 8 transactions for illustrating the model in figures 3, 4, 5, 6, 7 and 9.

Example E_2 : The second running example E_2 contains a simpler configuration of transactions, but is especially designed for illustrating the generated formula to be minimized by a quantum annealer. In E_2 , three transactions are to be distributed on two machines. The first transaction has a length of 2, the other two a length of 1. Furthermore, the second and third transactions are blocking each other in their execution. We use $R = 2$ for the maximum execution time (see Section 3.3.3 for determining R).

Overall we have the following configuration for the running example E_2 :

$$\begin{aligned} R &= 2 \\ T &= \{t_1, t_2, t_3\} \text{ with } |T| = 3 \\ M &= \{m_1, m_2\} \text{ with } |M| = 2 \\ O &= \{(t_2, t_3)\} \\ l_1 &= 2, l_2 = 1, l_3 = 1 \\ r_1 &= 0, r_2 = 1, r_3 = 1 \end{aligned}$$

²As discussed in Section 2.1.2 and by using the approach in [23], transactions can be also processed, which do not have a fixed set of required locks, by introducing an additional processing phase before transaction start.

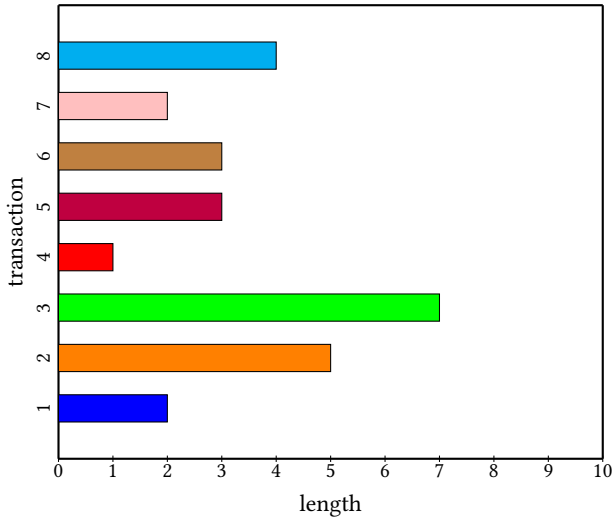


Figure 3: $n = 8$ transactions with their respective lengths (Example E_1)

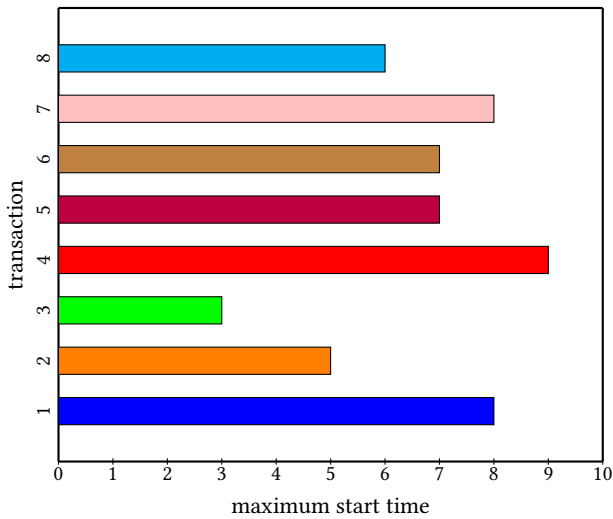


Figure 4: $n = 8$ transactions with their respective maximum start times (Example E_1)

3.2 Formulation as QUBO-problem

A QUBO-problem consists of binary variables that occur weighted in linear or quadratic terms. The binary variables

$$X_{i,j,s} \text{ for } 1 \leq i \leq n, 1 \leq j \leq k, 0 \leq s \leq r_i$$

contain the value 1 if transaction t_i is started at time s on machine m_j , otherwise 0. For a valid schedule n of the variables must hold the value 1 and all others 0, so that for each transaction the respective start time is expressed. The solution to this problem is the distribution of the transactions to the different machines (see Figure 7).

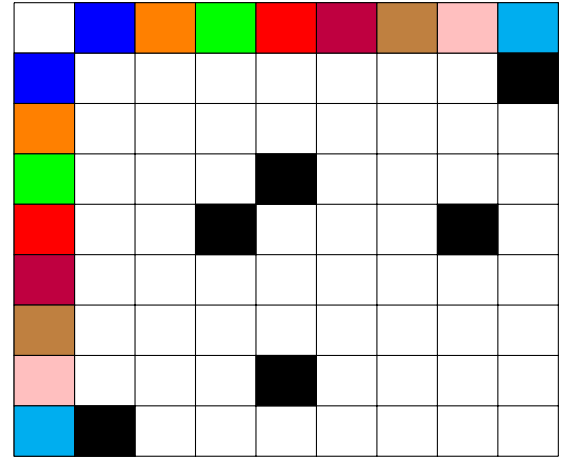


Figure 5: Black fields indicate blocking transactions (Example E_1)

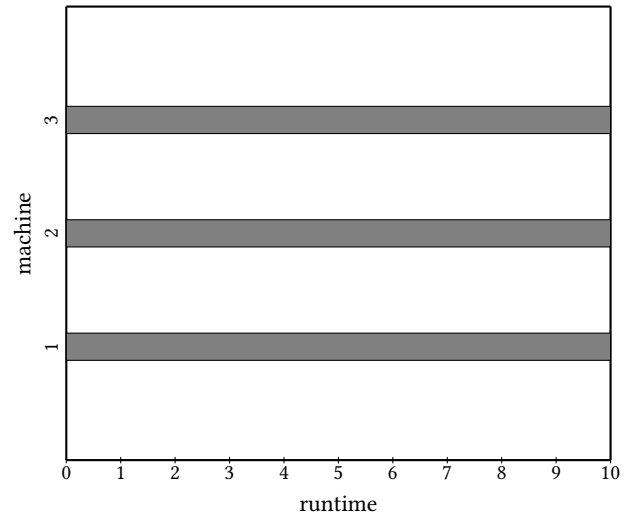


Figure 6: Transactions are scheduled on three machines, the maximum runtime here is $R = 10$ (Example E_1)

3.3 Valid solution

Constraints for a valid schedule are now formulated in such a way that the resulting formula takes on high values whenever the constraints for validity are not satisfied and low values whenever they are satisfied. Three constraints must be formulated so that minimizing the formula guarantees a valid schedule:

- A: Each transaction starts exactly once,
- B: two or more transactions cannot be executed at the same time on the same machine, and
- C: transactions that block each other cannot be executed at the same time.

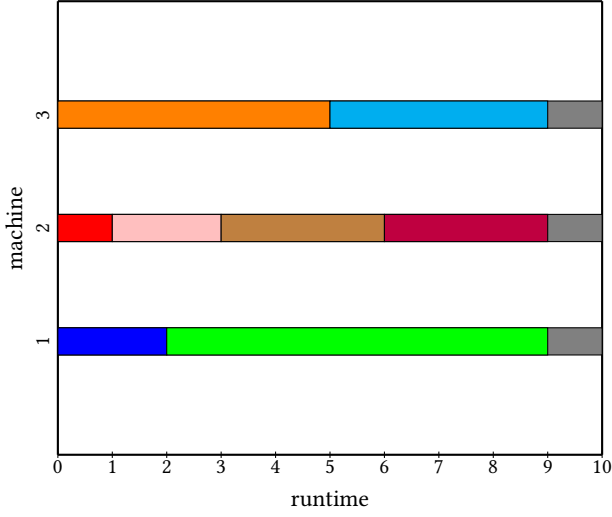


Figure 7: Binary variables set for this schedule:

$X_{110}, X_{312}, X_{420}, X_{721}, X_{623}, X_{526}, X_{230}, X_{835}$ (Example E_1)

To ensure that each transaction starts exactly once, it may only start once across all machines and possible start times:

$$A = \sum_{i=1}^n \left(\sum_{j=1}^k \sum_{s=0}^{r_i} X_{i,j,s} - 1 \right)^2$$

transactions machines start times

For the example configuration E_2 of Section 3.1.1:

$$A = (X_{1,1,0} + X_{1,2,0} - 1)^2 + (X_{2,1,0} + X_{2,1,1} + X_{2,2,0} + X_{2,2,1} - 1)^2 + (X_{3,1,0} + X_{3,1,1} + X_{3,2,0} + X_{3,2,1} - 1)^2$$

A holds the value 0 if each transaction starts exactly once, a higher one otherwise. If for a transaction the rear term in brackets is multiplied with $k \cdot r_i$ variables, only terms with variables appear and offsets of 1 = $-1 \cdot -1$. Hence we obtain n offsets for n variables. Since a QUBO-problem consists solely of linear and quadratic terms, the offset has no place there and is ignored. Hence during running the problem on a quantum annealer, A finally retrieves only the value $-n$ and the offset of n is ignored. After the final formula has been minimized and a variable assignment has been found, the offset can of course be added again, but it does not play any role in solving the problem.

To ensure that transactions do not run at the same time on the same machine, the runtime is calculated in the following way using the start time and the length of the transactions:

$$B = \sum_{j=1}^k \sum_{i_1=1}^{n-1} \sum_{s_1=0}^{r_{i_1}} \sum_{i_2=i_1+1}^n \sum_{s_2=q}^p X_{i_1,j,s_1} X_{i_2,j,s_2}$$

machines transactions without t_n start times remaining transactions invalid start times

$$\text{for } q = \max\{0, s_1 - l_{i_2} + 1\}, p = \min\{s_1 + l_{i_1}, r_{i_2}\}$$

For the example configuration E_2 of Section 3.1.1:

$$B = X_{1,1,0}X_{2,1,0} + X_{1,1,0}X_{2,1,1} + X_{1,1,0}X_{3,1,0} + X_{1,1,0}X_{3,1,1} + X_{2,1,0}X_{3,1,0} + X_{2,1,1}X_{3,1,1} + X_{1,2,0}X_{2,2,0} + X_{1,2,0}X_{2,2,1} + X_{1,2,0}X_{3,2,0} + X_{1,2,0}X_{3,2,1} + X_{2,2,0}X_{3,2,0} + X_{2,2,1}X_{3,2,1}$$

If no transactions are running at the same time on the same machine, at most one of the variables pairwise takes the value 1 and B overall takes the value 0. For each machine, transaction and associated start time, all invalid start times of all remaining transactions are calculated and this combination is added to the formula. B reaches the maximum, i.e., exactly the value of all sums of the formula, in the case of pairwise both variables take the value 1. The values q and p delimit the range in which two transactions overlap in their runtimes for certain start times (see Figure 8).

To avoid transactions that block each other being executed at the same time, similar constraints are established as for B :

$$C = \sum_{\{t_{i_1}, t_{i_2}\} \in O} \sum_{j_1=1}^k \sum_{s_1=0}^{r_{i_1}} \sum_{j_2 \in J} \sum_{s_2=q}^p X_{i_1,j_1,s_1} X_{i_2,j_2,s_2}$$

blocking transactions machines start times remaining machines invalid start times

for $J = \{1, \dots, k\} \setminus \{j_1\}$,

$$q = \max\{0, s_1 - l_{i_2} + 1\}, p = \min\{s_1 + l_{i_1}, r_{i_2}\}$$

For the example configuration E_2 of Section 3.1.1:

$$C = X_{2,1,0}X_{3,2,0} + X_{2,1,1}X_{3,2,1} + X_{2,2,0}X_{3,1,0} + X_{2,2,1}X_{3,1,1}$$

Similar to B , C takes the value 0, if no transactions that block each other are executed at the same time and so again only one of the variables pairwise takes the value 1. For all pairs of blocking transactions, the different invalid start combinations are determined and added to the formula. C reaches the maximum, i.e., exactly the value of all sums of the formula, in the case of pairwise both variables take the value 1.

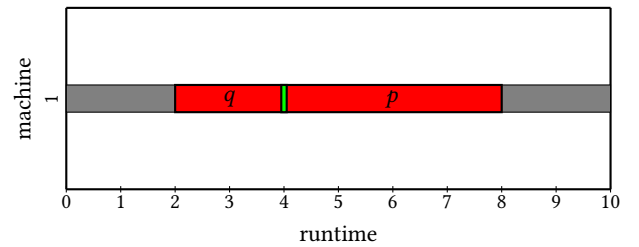


Figure 8: The green line represents a starting time 4 of a transaction of length 4 on a machine. The value q indicates that a second transaction of length 3 must not be started in the red area in front of the green line, because otherwise the runtimes will overlap. The value p expresses the time until when the first transaction runs, so that no other transactions may be started in the second red area either.

3.3.1 Optimal solution. For an optimal solution, the variables are now weighted so that minimizing the formula requires the earliest possible start time for each transaction. Therefore the calculated end time is weighted so that its weight exceeds the sum of all weights of all machines for each smaller end time. This is necessary because otherwise many short transactions that are started early would keep the weight lower than long transactions. As a result, long transactions with a different weight distribution would only be started at the end and thus the actual goal of minimizing the maximum execution time would not be achieved. The weight for an end time $s + l_i$ looks like this (see Figure 9):

$$w_{s+l_i} = \frac{(k+1)^{s+l_i-1}}{(k+1)^R}$$

By dividing by $(k+1)^R$, $w_{s+l_i} \in (0, 1)$ applies, which requires that a valid solution is preferred to an optimal one, since the weights in A , B and C always assume the value $|1|$ or higher. This results in the following formula:

$$D = \sum_{i=1}^n \sum_{j=1}^k \sum_{s=0}^{r_i} w_{s+l_i} X_{i,j,s}$$

For the example configuration E_2 of Section 3.1.1:

$$\begin{aligned} D = & \frac{3}{9}X_{1,1,0} + \frac{3}{9}X_{1,2,0} \\ & + \frac{1}{9}X_{2,1,0} + \frac{3}{9}X_{2,1,1} + \frac{1}{9}X_{2,2,0} + \frac{3}{9}X_{2,2,1} \\ & + \frac{1}{9}X_{3,1,0} + \frac{3}{9}X_{3,1,1} + \frac{1}{9}X_{3,2,0} + \frac{3}{9}X_{3,2,1} \end{aligned}$$

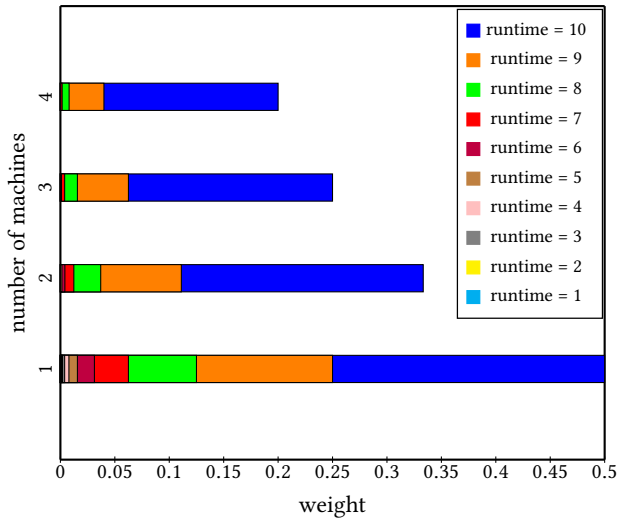


Figure 9: Display of weights for various numbers of machines and end times (Example E_1)

3.3.2 Total solution. Together A , B , C and D form the actual formula, the QUBO-problem, the solution of which is the solution of the actual problem, a valid and optimal distribution of the transactions to the different machines, so that if possible there are no idle times and if possible each transaction has no waiting time:

$$P = A + B + C + D$$

A valid solution always assumes the value $-n$ for $A + B + C$ and is increased by the value of D whenever an optimal solution is reached.

For the example configuration E_2 of Section 3.1.1, the formula P is minimized if all three constraints for a valid schedule are satisfied and the weights across all variables are minimal. In this example, the formula is minimized by four different variable assignments, all of which represent a valid schedule (here, only the variables with the value 1 are specified, the rest take the value 0 accordingly):

$$\begin{aligned} & X_{1,1,0}, X_{2,2,0}, X_{3,2,1} \\ & X_{1,1,0}, X_{2,2,1}, X_{3,2,0} \\ & X_{1,2,0}, X_{2,1,0}, X_{3,1,1} \\ & X_{1,2,0}, X_{2,1,1}, X_{3,1,0} \end{aligned}$$

The solution represents which transaction is started when and on which machine. P assumes the following value for all four different variable assignments:

$$P = A + B + C + D = -3 + 0 + 0 + \frac{7}{9} = -2\frac{2}{9}$$

If the offset is added (optional), then the result is:

$$P = A + B + C + D = -3 + 0 + 0 + \frac{7}{9} + 3 = \frac{7}{9}$$

Both solutions are correct, for the purpose of simplicity the offset is omitted here. The solution is illustrated in Figure 10.

3.3.3 Minimization of execution time. The (maximal) execution time R is a critical parameter during generation of the formula and its solving: If R is too large, then we need too many unnecessary variables, which increases the problem size and hence also the time of preprocessing (but we still retrieve the correct solution). If R is too small, then we do not obtain any solution. In the latter case, we need to rerun the formula generation and solving it on a quantum annealer with an increased R .

The execution time R should therefore be reasonably estimated in advance to avoid unnecessarily many runs on a quantum annealer. Since a quantum annealer executes many runs of the same problem, by slowly increasing the previously estimated execution time, the minimum execution time allowed by a valid schedule can be determined. The lower bound of the execution time is obviously the sum of all lengths divided by the number of machines:

$$R = \frac{\sum_{i=1}^n l_i}{k}$$

This value can be estimated even better when considering the length of transactions. Two transactions with lengths 1 and 9 cannot be executed by two machines in time $R = (1 + 9)/2 = 5$. A more reasonable estimation is:

$$R = \max\left\{\frac{\sum_{i=1}^n l_i}{k}, \max_{i \in \{1, \dots, n\}} l_i\right\}$$

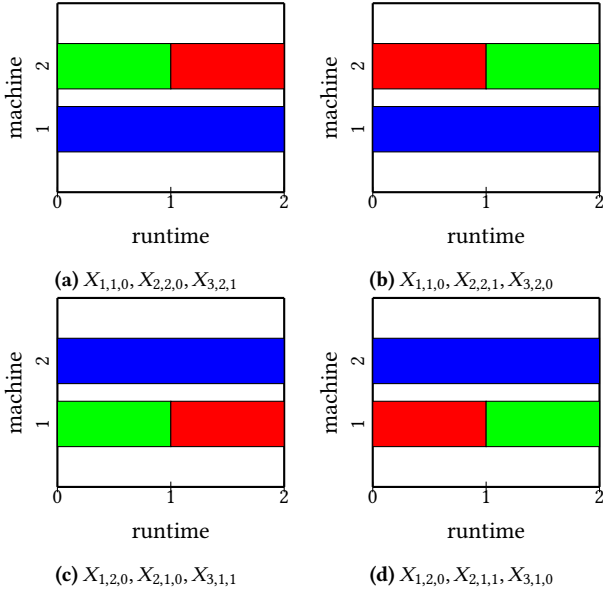


Figure 10: Different Schedules (transaction 1 in blue, transaction 2 in green and transaction 3 in red) for example configuration E_2

For the example configuration E_2 of Section 3.1.1:

$$R = \max\left\{\frac{4}{2}, 2\right\} = 2$$

If there is one very long transaction and many short ones, the maximum execution time is obviously defined by the length of the very long transaction. By slowly increasing R the minimum execution time and a suitable schedule are found. A valid schedule is identified by its value, since this value does not differ significantly from $-n$, but is significantly larger for an invalid schedule.

4 IMPLEMENTATION OF THE QUBO-PROBLEM

We implement the QUBO-problem described in Section 3.2 by using the Ocean Software of the company D-Wave [2]. The Ocean Software consists of a set of tools that help to formulate a problem for quantum annealers and solves the problem using either a classical computer or a quantum annealer. By only adapting the code for communicating with the quantum annealer, the same code is runnable on a classical computer as well as on a quantum annealer with minimal code modification. The software provides tools for communication with the quantum annealer, for problem formulation through various constraints, for problem solving through various approaches, for embedding the problem on the quantum annealer and many more. Additionally, the software “PyQUBO” [5] is used, which allows to design QUBO formulations from mathematical expressions. Since both software is developed for Python, the implementation is also written in Python. The packages “dimod” [3] (for supporting QUBO models) and “dwave-neal” [4] (for comparing the runtimes of quantum with those of the simulated annealing)

are additionally used. The implementation calculates the QUBO-problem from a list of transactions and their lengths, a number of machines and a list of blocking transactions. The execution time R for these transactions is estimated in advance according to Section 3.3.3. The resulting QUBO-problem is precisely solvable by testing all possible variable assignments and thus those with the lowest total value represent the optimal and valid solution. For determining a solution on a classical computer the Simulated-Annealing-Solver [4], one of the Ocean Software tools, is used. For quantum annealing the D-Wave Quantum-Annealer [9] offered as cloud service is used.

5 EVALUATION

In this section, we analyze the complexity (see Section 5.1) in terms of preprocessing time (see Section 5.1.1) and required qubits (see Section 5.1.2), and experimental results comparing simulated with quantum annealing (see Section 5.2).

5.1 Formal analysis

A time analysis for quantum computers as a function of problem variables such as number of transactions or number of machines is generally not possible, since for quantum computers the number of annealing runs as well as the times per annealing run and the times for readout can be determined in advance.

We first determine the number of required qubits and the time of preprocessing in terms of a complexity analysis in Section 5.1.1. The problem sizes remain n for the number of transactions, k for the number of machines and R for the maximum execution time.

5.1.1 Preprocessing time. $O(n \cdot k \cdot R)$ binary variables are used within the formula for distributing transactions to k machines to be minimized. A and D contain $O(n \cdot k \cdot R)$ terms, B contains $O(n^2 \cdot k \cdot R^2)$ terms and C in case all transactions block each other $O((n \cdot k \cdot R)^2)$ terms. Calculating the weights for D is performed in constant time. For the worst case, we overall achieve a quadratic time for preprocessing depending on the problem sizes:

$$O((n \cdot k \cdot R)^2)$$

For cases, where the number of conflicting transactions is at most linear to n (which might be the typical case for transaction workloads in operational systems), the number of terms is dominated by the number of terms in B (assuming $n > k$) and is hence $O(n^2 \cdot k \cdot R^2)$.

5.1.2 Required qubits. In quantum annealers from D-Wave, qubits are arranged in the Chimera structure, which simply states that each qubit is entangled with a maximum of six other qubits. If the binary variables occur in quadratic terms, the entanglements are weighted accordingly. Since only entanglements can be weighted, but no single qubits and binary variables usually occur weighted with more than six other variables together, several qubits are defined as one variable. The individual weights are now distributed over the entanglements of the qubits of a variable. For the entanglements one qubit per variable is selected and the entanglement is weighted. Consequently, the number of qubits required depends on how many connections the binary variables have with each other. A and D do not contain quadratic terms and are therefore not important. In B each variable is connected to a maximum of $O(n \cdot R)$ others, in C

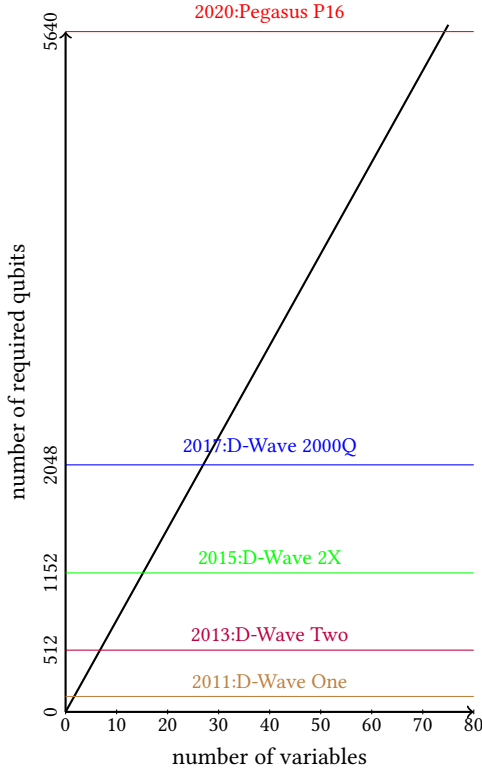


Figure 11: Required qubits depending on the number of variables and available qubits of D-Wave Quantum-Annealers

to a maximum of $O(n \cdot k \cdot R)$. The number of required qubits thus increases quadratically dependent on the problem sizes:

$$O((n \cdot k \cdot R)^2)$$

The maximum number of variables is thus limited by the root of the number of qubits. Figure 11 shows the number of required qubits as a function of the number of variables, where the horizontal lines represent the numbers of qubits of available and future quantum annealers.

5.2 Experimental analysis

For the experimental analysis, the time of solving is measured using simulated annealing and quantum annealing. We present the details about the test configurations (i.e., number of cores, maximum execution times, transactions and conflicts between them, their start times and lengths) in the Appendix.

For quantum annealing we use the cloud-service available at [9], which offers free access to a D-Wave 2000Q quantum computer for some time. The offered free time was enough to run the experimental evaluation described in Section 5.2.1.

The experiments for preprocessing and simulated annealing are run on an i7-4510U dual core CPU with 2.0 GHz and 8 GB main memory running Windows 10. Every simulated annealing run and quantum annealing run was measured a hundred times and the lowest value is determined as an (almost) optimal solution. We

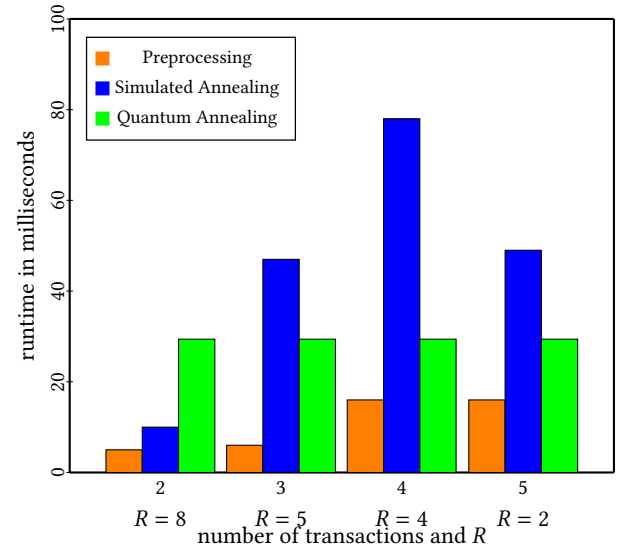


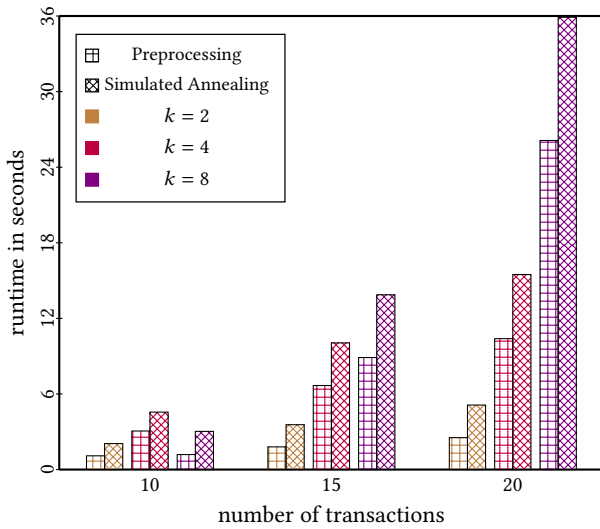
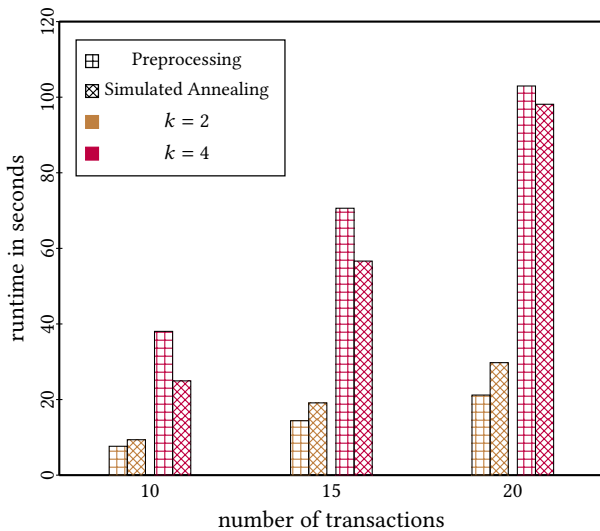
Figure 12: Runtimes of simulated annealing versus quantum annealing for $k = 2$

present the overall total time of these hundred runs in the following sections.

5.2.1 Runtimes Quantum Annealing. Due to the limited number of qubits only small problem instances could be measured by quantum annealing (see Figure 12). The preprocessing time for generating the formula is the same for simulated as well as for quantum annealing, as both take the formula as input and find a minimal solution for it.

For every quantum annealing run the determined runtime was 20 microseconds as well as 274 microseconds for the readout. Measuring a hundred times will then take 29.4 milliseconds. While for the runtime of two transactions simulated annealing is still faster than quantum annealing, already problem sizes with 3 transactions and above are faster with quantum annealing with an observed speedup up to approx. 2.65. Once quantum annealers supporting more qubits are available, larger problem sizes can be solved with quantum annealing. As the times for quantum annealing and readout are constant, but simulated annealing runs become slower for larger problem sizes due to the more complex evaluation of the formula, larger speedups will be probably achieved when running on future quantum annealers.

5.2.2 Runtimes Simulated Annealing. For simulated annealing, we don't have limitations concerning the number of variables used in the formula to minimize. Hence we measure the runtimes of simulated annealing for larger problem sizes (see Figures 13 and 14). With a total of around 30 milliseconds the (constant) runtime of pure optimization by the quantum annealer is significantly better than that of simulated annealing even for relatively small problem instances. Assuming that also future quantum annealers will have approximately the same (constant) runtimes, speedups of quantum in comparison to simulated annealing up to 3300 for the problem sizes in Figures 13 and 14 are achievable.

Figure 13: Runtimes for $R = 20$ Figure 14: Runtimes for $R = 50$

6 SUMMARY AND CONCLUSIONS

In this paper, we show how to transform the problem of distributing transactions to arbitrary machines into an optimization problem that can be solved by quantum computers. In more detail, we transform a corresponding problem instance into a QUBO-problem to be run on a quantum annealer that is a special type of quantum computer for solving QUBO-problems. In the evaluation, the number of qubits required for the QUBO-problem of distributing transactions and the time needed for the transformation are analyzed. Furthermore, we conduct an experimental analysis comparing the runtimes of a quantum annealer with the one of simulated annealing on a classical computer. We already achieved speedups of up to over 2.6. However, the number of qubits of current quantum annealers has

so far limited the number of possible problem sizes to rather small ones, which promises higher speedups for future generations of quantum annealers supporting many more qubits. Thus a real use of quantum computers for optimization problems of this kind is likely to continue in the future.

In our future work we will investigate how to improve e.g. the preprocessing time by indexing subformulas in some kind of cache for already determined transaction setups, such that formulas can be built based on previous ones by just iterating through the pairs of blocking transactions. Furthermore, we may also consider to accelerate other transaction models and synchronization problems as well as other areas of database research by quantum computers.

REFERENCES

- [1] 2017. The D-Wave 2000Q™ System. <https://www.dwavesys.com/d-wave-two-system>
- [2] 2017. D-Wave's Ocean Software. <https://ocean.dwavesys.com>
- [3] 2017. dimod. <https://github.com/dwavesystems/dimod>
- [4] 2017. dwave-neal. <https://github.com/dwavesystems/dwave-neal>
- [5] 2017. PyQUBO. <https://github.com/recruit-communications/pyqubo>
- [6] Adriano Barenco, David Deutsch, Artur Ekert, and Richard Jozsa. 1995. Conditional quantum dynamics and logic gates. *Physical Review Letters* 74, 20 (1995), 4083.
- [7] Carlos Barrón-Romero. 2015. Classical and Quantum Algorithms for the Boolean Satisfiability Problem. *arXiv preprint arXiv:1510.02682* (2015).
- [8] Thomas M Connolly and Carolyn E Begg. 2005. *Database systems: a practical approach to design, implementation, and management*. Pearson Education.
- [9] D-Wave Systems. 2020. Take the Leap. <https://www.dwavesys.com/take-leap>
- [10] Edward Farhi, Jeffrey Goldstone, Sam Gutmann, and Michael Sipser. 2000. Quantum computation by adiabatic evolution. *arXiv preprint quant-ph/0001106* (2000).
- [11] M. R. Garey, D. S. Johnson, and Ravi Sethi. 1976. The Complexity of Flowshop and Jobshop Scheduling. *Mathematics of Operations Research* 1, 2 (1976), 117–129.
- [12] Fred Glover, Gary Kochenberger, and Yu Du. 2019. Quantum Bridge Analytics I: a tutorial on formulating and using QUBO models. *4OR* 17, 4 (2019), 335–371.
- [13] Sven Groppe. 2020. Emergent models, frameworks, and hardware technologies for Big data analytics. *The Journal of Supercomputing* 76, 3 (2020), 1800–1827. <https://doi.org/10.1007/s11227-018-2277-x>
- [14] Sven Groppe and Jinghua Groppe. 2020. Hybrid Multi-Model Multi-Platform (HM3P) Databases. In *Proceedings of the 9th International Conference on Data Science, Technology and Applications (DATA)*.
- [15] Lov K Grover. 1996. A fast quantum mechanical algorithm for database search. *arXiv preprint quant-ph/9605043* (1996).
- [16] Lov K Grover. 1997. Quantum computers can search arbitrarily large databases by a single query. *Physical review letters* 79, 23 (1997), 4709.
- [17] Tad Hogg. 2003. Adiabatic quantum computing for random satisfiability problems. *Physical Review A* 67, 2 (2003), 022314.
- [18] Anastasia Marchenkova. 2016. What's the difference between quantum annealing and universal gate quantum computers? <https://medium.com/quantum-bits>
- [19] Martin Pincus. 1970. Letter to the Editor-A Monte Carlo Method for the Approximate Solution of Certain Types of Constrained Optimization Problems. *Operations Research* 18, 6 (1970), 1225–1228.
- [20] Sudip Roy, Lucja Kot, and Christoph Koch. 2013. Quantum databases. In *Proc. CIDR*.
- [21] Peter W Shor. 1994. Algorithms for quantum computation: Discrete logarithms and factoring. In *Proceedings 35th annual symposium on foundations of computer science*. Ieee, 124–134.
- [22] Tobias Stollenwerk and Achim Basermann. 2016. Experiences with scheduling problems on adiabatic quantum computers. In *Proceedings of the 1st International Workshop on Post-Moore Era Supercomputing (PMES), Future Technologies Group Technical report FTGTR-2016-11*. 45–46.
- [23] A. Thomson, T. Diamond, S.-C. Weng, K. Ren, P. Shao, and D. J. Abadi. 2012. Calvin: Fast Distributed Transactions for Partitioned Database Systems. In *Proceedings of the ACM SIGMOD International Conference on Management of Data (SIGMOD)* (Scottsdale, Arizona, USA) (SIGMOD '12). 1–12. <https://doi.org/10.1145/2213836.2213838>
- [24] Immanuel Trummer and Christoph Koch. 2016. Multiple Query Optimization on the D-Wave 2X Adiabatic Quantum Computer. *Proc. VLDB Endow.* 9, 9 (May 2016), 648–659. <https://doi.org/10.14778/2947618.2947621>
- [25] Davide Venturelli, Dominic J. J. Marchand, and Galo Rojo. 2015. Quantum Annealing Implementation of Job-Shop Scheduling. *arXiv* (2015). arXiv:1506.08479 [quant-ph] <https://arxiv.org/abs/1506.08479>

APPENDIX

We present the test configurations for figures 12,13 and 14 in the following tables. For each test configuration, we assume $T = \{t_1, \dots, t_n\}$ and $M = \{m_1, \dots, m_k\}$. The last column of the following table contains the number of required variables.

Fig.	k	n	R	O	$l_1, \dots, l_n$	$r_1, \dots, r_n$	req. var.
12	2	2	8	$\{\}$	8, 4	0, 4	8
		3	5	$\{(t_1, t_3)\}$	4, 5, 1	1, 0, 4	10
		4	4	$\{(t_2, t_4)\}$	3, 2, 1, 2	1, 2, 3, 2	16
		5	2	$\{(t_1, t_2), (t_4, t_5)\}$	1, 1, 1, 1, 1	1, 1, 1, 1, 1	10
13	2	10		$\{(t_1, t_3), (t_3, t_4), (t_7, t_8)\}$	5, 7, 1, 1, 3, 4, 4, 2, 8, 5	15, 13, 19, 19, 15, 16, 16, 18, 12, 15	320
	4			$\{(t_1, t_2), (t_9, t_{10})\}$	10, 14, 8, 5, 4, 7, 6, 10, 3, 13	10, 6, 12, 15, 16, 13, 14, 10, 17, 7	480
	8			$\{(t_1, t_2), (t_3, t_4), (t_5, t_6)\}$	16, 20, 5, 15, 4, 6, 14, 10, 6, 12	4, 0, 15, 5, 16, 14, 6, 10, 14, 8	736
	2	15		$\{(t_2, t_3), (t_4, t_6), (t_5, t_{10}), (t_1, t_{15})\}$	4, 2, 3, 1, 4, 6, 2, 1, 5, 1, 1, 3, 2, 4, 1	16, 18, 17, 19, 16, 14, 18, 19, 15, 19, 19, 17, 18, 16, 19	520
	4			$\{(t_1, t_2), (t_3, t_4), (t_7, t_8)\}$	6, 4, 3, 1, 2, 8, 7, 10, 6, 11, 1, 3, 6, 4, 8	14, 16, 17, 19, 18, 12, 13, 10, 14, 9, 19, 17, 14, 16, 12	880
	8			$\{(t_2, t_4), (t_3, t_6), (t_3, t_{14})\}$	9, 12, 5, 3, 8, 14, 3, 7, 8, 1, 2, 4, 20, 1, 3	11, 8, 15, 17, 12, 6, 17, 13, 12, 19, 18, 16, 0, 19, 17	1600
	2	20		$\{(t_5, t_{12}), (t_6, t_{18}), (t_7, t_{20}), (t_8, t_{20})\}$	1, 1, 1, 2, 4, 1, 1, 2, 1, 1, 2, 2, 2, 1, 1, 1, 1, 6, 5, 4	19, 19, 19, 18, 16, 19, 19, 18, 19, 19, 18, 18, 18, 19, 19, 19, 19, 14, 15, 16	720
	4			$\{(t_1, t_2), (t_2, t_{17}), (t_5, t_9)\}$	3, 4, 1, 2, 4, 1, 1, 7, 1, 10, 1, 9, 5, 3, 8, 5, 2, 7, 1, 8	17, 16, 19, 18, 16, 19, 19, 13, 19, 10, 19, 11, 15, 17, 12, 15, 18, 13, 19, 12	1268
	8			$\{(t_2, t_4), (t_8, t_{19}), (t_{11}, t_{13})\}$	7, 16, 8, 4, 11, 4, 6, 5, 7, 12, 19, 13, 1, 9, 1, 2, 3, 4, 15, 14	13, 4, 12, 16, 9, 16, 14, 15, 13, 8, 1, 7, 19, 11, 19, 18, 17, 16, 5, 6	1912
14	2	10		$\{(t_2, t_{10}), (t_3, t_4)\}$	15, 17, 10, 10, 4, 9, 8, 12, 2, 13	35, 33, 40, 40, 46, 41, 42, 38, 48, 37	800
	4			$\{(t_1, t_2), (t_9, t_{10})\}$	20, 21, 12, 25, 24, 16, 27, 15, 18, 22	30, 29, 38, 25, 26, 34, 23, 35, 32, 28	1200
	2			$\{(t_1, t_2), (t_3, t_{11}), (t_5, t_{10})\}$	6, 7, 3, 1, 12, 8, 7, 10, 4, 11, 1, 13, 6, 4, 8	44, 43, 47, 49, 38, 42, 43, 40, 46, 39, 49, 37, 44, 46, 42	1298
	4	15	50	$\{(t_1, t_3), (t_3, t_4), (t_7, t_9), (t_{11}, t_{12})\}$	15, 12, 9, 1, 12, 22, 23, 14, 9, 8, 14, 17, 24, 13, 7	35, 38, 41, 49, 38, 28, 27, 36, 41, 42, 36, 33, 26, 37, 43	2200
	2			$\{(t_4, t_5), (t_5, t_6), (t_{12}, t_{13}), (t_{17}, t_{18})\}$	3, 4, 1, 2, 4, 1, 10, 7, 1, 11, 1, 9, 8, 3, 8, 5, 2, 7, 1, 12	47, 46, 49, 48, 46, 49, 40, 43, 49, 39, 49, 41, 42, 48, 42, 45, 48, 43, 49, 38	1800
	4			$\{(t_3, t_{17}), (t_5, t_7), (t_6, t_8)\}$	12, 17, 19, 4, 11, 22, 3, 14, 23, 2, 9, 12, 2, 12, 12, 4, 5, 2, 3, 14	38, 33, 31, 46, 39, 28, 47, 36, 27, 48, 41, 38, 48, 38, 38, 46, 45, 48, 47, 36	3192